

can be sampled by picking a pair of uniform deviates (s, t) and then mapping them into (x, y) by a linear transformation. Another good choice, V_3 , can be sampled by, first, using a uniform deviate to choose between the left and right rectangular subregions (in proportion to their respective areas!) and, then, using two more deviates to pick a point inside the chosen rectangle.

Let's create an object that embodies the general scheme described. (We will discuss the implementing code later.) The general idea is to create an `MCIntegrate` object by providing (as constructor arguments) the following items:

- a vector `xlo` of lower limits of the coordinates for the rectangular box to be sampled
- a vector `xhi` of upper limits of the coordinates for the rectangular box to be sampled
- a vector-valued function `funcs` that returns as its components one or more functions that we want to integrate simultaneously
- a boolean function that returns whether a point is in the (possibly complicated) region W that we want to integrate; the point will already be within the region V defined by `xlo` and `xhi`
- a mapping function to be discussed below, or `NULL` if there is no mapping function or if your attention span is too short
- a seed for the random number generator

The object `MCIntegrate` has this structure.

```
struct MCIntegrate { mcintegrate.h
    Object for Monte Carlo integration of one or more functions in an ndim-dimensional region.
    Int ndim,nfun,n;           Number of dimensions, functions, and points sampled.
    VecDoub ff,fferr;          Answers: The integrals and their standard errors.
    VecDoub xlo,xhi,x,xx,fn,sf,sferr;
    Doub vol;                  Volume of the box  $V$ .

    VecDoub (*funcsp)(const VecDoub &);           Pointers to the user-supplied functions.
    VecDoub (*xmapp)(const VecDoub &);
    Bool (*inregionp)(const VecDoub &);
    Ran ran;                                       Random number generator.

    MCIntegrate(const VecDoub &xlow, const VecDoub &xhigh,
    VecDoub funcs(const VecDoub &), Bool inregion(const VecDoub &),
    VecDoub xmap(const VecDoub &), Int ranseed);
    Constructor. The arguments are in the order described in the itemized list above.

    void step(Int nstep);
    Sample an additional nstep points, accumulating the various sums.

    void calcanswers();
    Calculate answers ff and fferr using the current sums.
};
```

The member function `step` adds sample points, the number of which is given by its argument. The member function `calcanswers` updates the vectors `ff` and `fferr`, which contain respectively the estimated Monte Carlo integrals of the functions and the errors on these estimates. You can examine these values, and then, if you want, call `step` and `calcanswers` again to further reduce the errors.

A worked example will show the underlying simplicity of the method. Suppose that we want to find the weight and the position of the center of mass of an object of

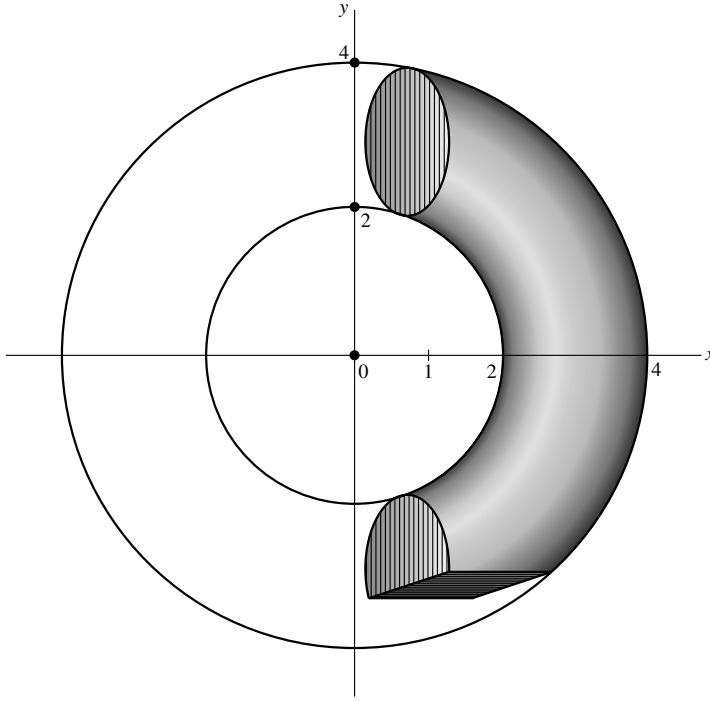


Figure 7.7.2. Example of Monte Carlo integration (see text). The region of interest is a piece of a torus, bounded by the intersection of two planes. The limits of integration of the region cannot easily be written in analytically closed form, so Monte Carlo is a useful technique.

complicated shape, namely the intersection of a torus with the faces of a large box. In particular, let the object be defined by the three simultaneous conditions:

$$z^2 + \left(\sqrt{x^2 + y^2} - 3 \right)^2 \leq 1 \quad (7.7.3)$$

(torus centered on the origin with major radius = 3, minor radius = 1)

$$x \geq 1 \quad y \geq -3 \quad (7.7.4)$$

(two faces of the box; see Figure 7.7.2). Suppose for the moment that the object has a constant density $\rho = 1$.

We want to estimate the following integrals over the interior of the complicated object:

$$\int \rho \, dx \, dy \, dz \quad \int x\rho \, dx \, dy \, dz \quad \int y\rho \, dx \, dy \, dz \quad \int z\rho \, dx \, dy \, dz \quad (7.7.5)$$

The coordinates of the center of mass will be the ratio of the latter three integrals (linear moments) to the first one (the weight).

To use the `MCintegrate` object, we first write functions that describe the integrands and the region of integration W inside the box V .

```
VecDoub torusfuncs(const VecDoub &x) {
```

```
mcintegrate.h
```

Return the integrands in equation (7.7.5), with $\rho = 1$.

```
    Doub den = 1.;
    VecDoub f(4);
    f[0] = den;
    for (Int i=1;i<4;i++) f[i] = x[i-1]*den;
    return f;
}
```

```
Bool torusregion(const VecDoub &x) {
```

Return the inequality (7.7.3).

```
    return SQR(x[2])+SQR(sqrt(SQR(x[0])+SQR(x[1]))-3.) <= 1.;
}
```

The code to actually do the integration is now quite simple,

```
VecDoub xlo(3), xhi(3);
xlo[0] = 1.; xhi[0] = 4.;
xlo[1] = -3.; xhi[1] = 4.;
xlo[2] = -1.; xhi[2] = 1.;
MCintegrate mymc(xlo,xhi,torusfuncs,torusregion,NULL,10201);
mymc.step(1000000);
mymc.calcanswers();
```

Here we've specified the box V by `xlo` and `xhi`, created an instance of `MCintegrate`, sampled a million times, and updated the answers `mymc.ff` and `mymc.fferr`, which can be accessed for printing or another use.

7.7.1 Change of Variables

A change of variable can often be extremely worthwhile in Monte Carlo integration. Suppose, for example, that we want to evaluate the same integrals, but for a piece of torus whose density is a strong function of z , in fact varying according to

$$\rho(x, y, z) = e^{5z} \quad (7.7.6)$$

One way to do this is, in `torusfuncs`, simply to replace the statement

```
Doub den = 1.;
```

by the statement

```
Doub den = exp(5.*x[2]);
```

This will work, but it is not the best way to proceed. Since (7.7.6) falls so rapidly to zero as z decreases (down to its lower limit -1), most sampled points contribute almost nothing to the sum of the weight or moments. These points are effectively wasted, almost as badly as those that fall outside of the region W . A change of variable, exactly as in the transformation methods of §7.3, solves this problem. Let

$$ds = e^{5z} dz \quad \text{so that} \quad s = \frac{1}{5}e^{5z}, \quad z = \frac{1}{5}\ln(5s) \quad (7.7.7)$$

Then $\rho dz = ds$, and the limits $-1 < z < 1$ become $.00135 < s < 29.682$.

The `MCintegrate` object knows that you might want to do this. If it sees an argument `xmap` that is not `NULL`, it will assume that the sampling region defined by `xlo` and `xhi` is not in physical space, but rather needs to be mapped into physical space before either the functions or the region boundary are calculated. Thus, to effect our change of variable, we *don't* need to modify `torusfuncs` or `torusregion`, but we *do* need to modify `xlo` and `xhi`, as well as supply the following function for the argument `xmap`: